

# Kerberos Bronze Bit Attack – Theory (CVE-2020-17049)

 [habr.com/ru/articles/756240](https://habr.com/ru/articles/756240)

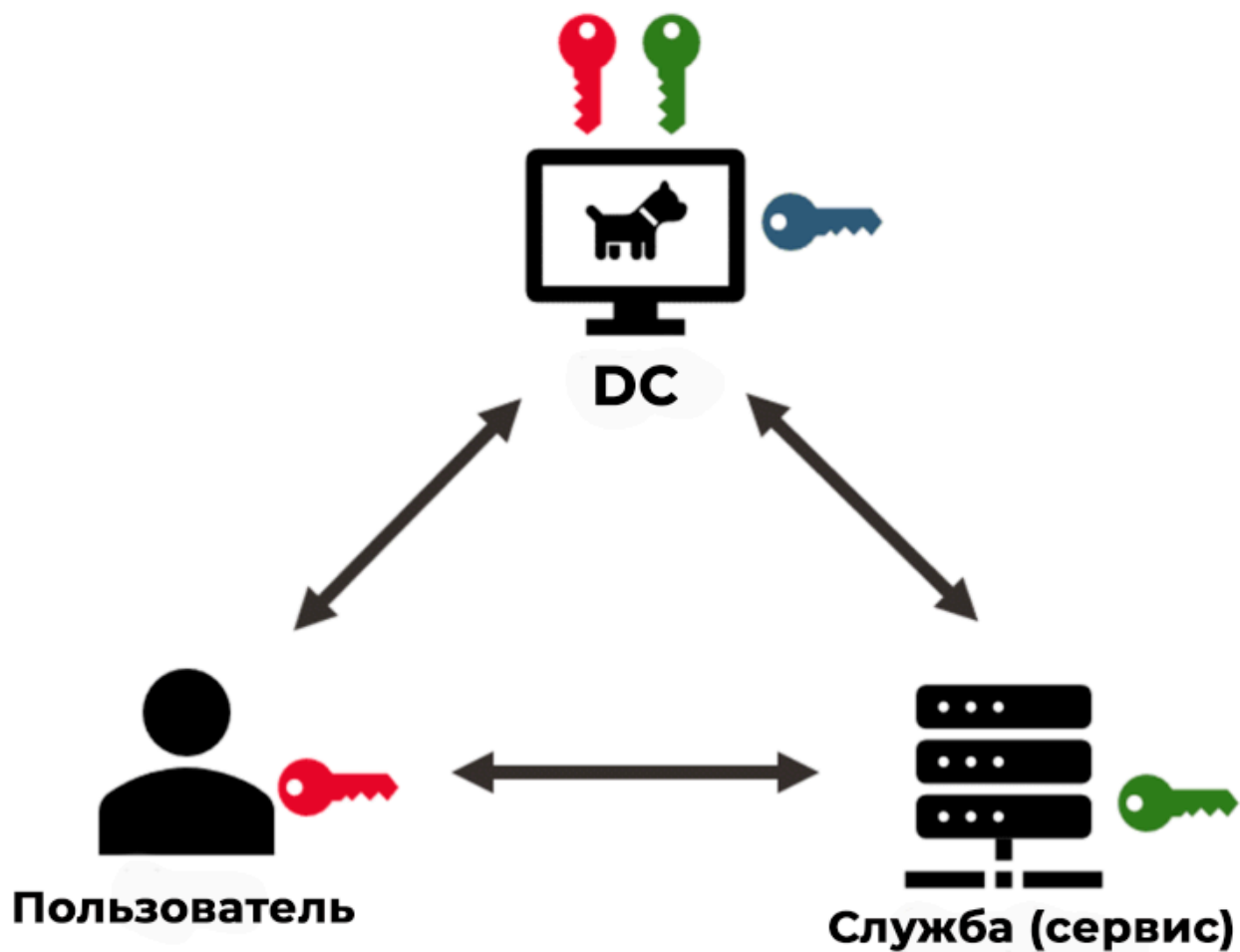
August 22, 2023

Эта статья будет посвящена уязвимости в расширениях для *Kerberos S4U2Self* и *S4U2Proxy*. Для полного понимания того, как работает уязвимость нужно знать, как происходит аутентификация в *Kerberos*, а также понимать такие процессы как делегирование.

***Дисклеймер: Все данные, предоставленные в данной статье, взяты из открытых источников, не призывают к действию и являются только лишь данными для ознакомления, и изучения механизмов используемых технологий.***

**Kerberos** – это протокол аутентификации пользователей, серверов и других ресурсов в домене. Протокол основан на симметричной криптографии, где каждый принципал имеет свой ключ. Этот ключ известен только самим принципалам и центру распределения ключей (*KDC – Key Distribution Center*). В его роли выступает контроллер домена (*DC, Domain Controller*).

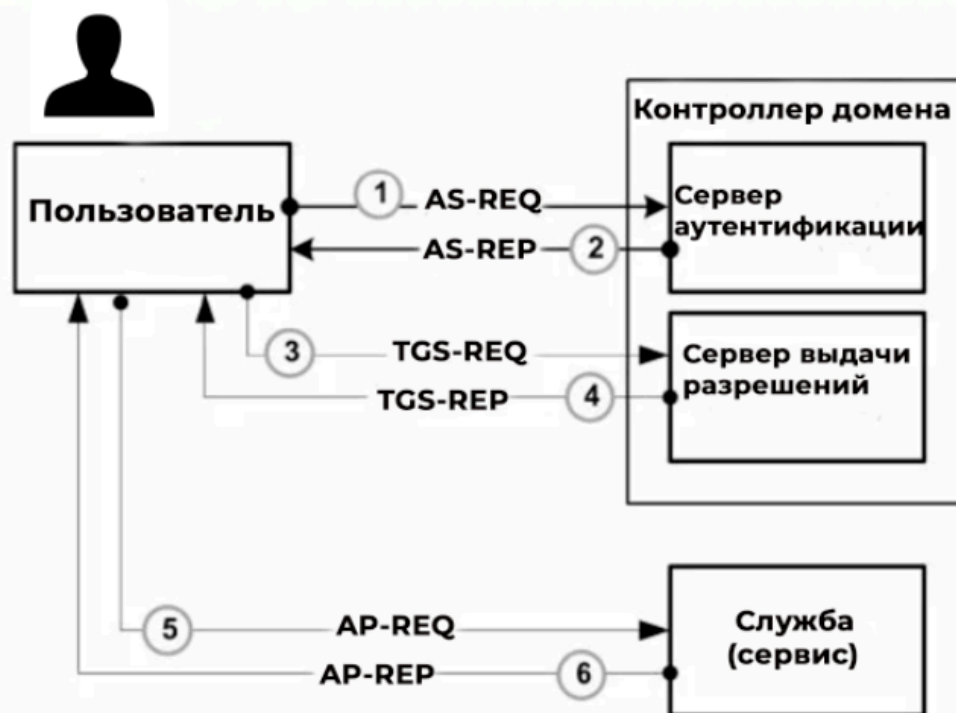
► Спойлер



Зная секретный ключ каждого пользователя, *KDC* облегчает аутентификацию пользователей, выдавая так называемые билеты ( *ticket* ). Билеты содержат информацию о пользователях и сервисах, а также ключи необходимые для общения с *DC*.

Итак, разберемся в деталях, как же работает аутентификация *Kerberos*, рассмотрев схему аутентификации.

# Аутентификация Kerberos



## AS\_Req

Первый этап работы протокола. Клиент приступает к аутентификации и отправляет на DC сообщение *AS\_Req* (*Authentication Service Request*). Это сообщение содержит:

- UPN пользователя (*User Principal Name*)
- Срок жизни билета (*timestamp*)
- Имя службы, к которой обращаются *krbtgt*

Наличие *timestamp* в *AS\_Req* называется предварительной аутентификацией пользователя. Ее можно отключить, и при ее отсутствии возможна атака типа *AS\_Req Roasting*.

## AS\_Rep

Сначала DC расшифровывает сообщение и проверяет метку времени, которая не должна отличаться более чем на 5 минут. Затем DC отправляет клиенту сообщение, состоящее из двух частей. Первая часть зашифрована секретом клиента и содержит:

- Метка времени (*timestamp*)
- Сессионный ключ для общения с DC
- Срок жизни TGT

Вторая часть зашифрована секретом *DC* и содержит тоже самое, что и первая часть вместе с *UPN* клиента. То есть фактически первая часть сообщения это сеансовый ключ, а вторая *TGT* (*ticket granting ticket*).

### ***TGS\_Req***

Получив *TGT*, «разрешение на выдачу разрешения», клиент приступает к запросу на получение разрешения общения с сервисом, к которому он хочет получить доступ. Клиент отправляет *DC* сообщение, которое содержит:

- *UPN* клиента и *timestamp*, зашифрованную сессионным ключом (аутентификатор)
- *SPN* сервиса
- *TGT*

Получив ответ от пользователя, *DC* проверяет не истек ли срок действия *TGT* и также сравнивает *UPN* из *TGT* и аутентификатора.

### ***TGS\_Rep***

Если все прошло успешно, то *DC* отправляет *TGS* зашифрованный на секрете сервиса, к которому хочет получить доступ клиент. Сам *TGS* состоит из:

- *SPN* сервиса
- *Timestamp*
- Срок жизни *TGS*

Далее пользователь проходит аутентификацию у сервиса, впоследствии получая к нему доступ. Там следуют *AP\_Req* запросы и *AP\_Rep* ответы.

## ***Делегирование в Kerberos***

Делегирование – предоставление доступа одному сервису к другому сервису от имени заданного пользователя. В протоколе *Kerberos* есть несколько видов делегирования. Начнем с **неограниченного делегирования**.

### ***Неограниченное делегирование***

При неограниченном делегировании сервис может обратиться к любому другому сервису от имени заданного пользователя. Но есть одна значительная деталь – у пользователя может установлен флаг *NOT\_DELEGATED* в атрибуте *UserAccountControl*, запрещающий обращение от указанного пользователя.

По умолчанию этот флаг отключен и устанавливается при активации опции «*Account is sensitive and cannot be delegated*» или при добавлении пользователя в группу *Protected Users*.

Чтобы учетная запись получила право на неограниченное делегирование, нужно в *UAC (UserAccountControl)* установить флаг *TRUSTED\_FOR\_DELEGATION*.

Теперь рассмотрим, как происходит процесс неограниченного делегирования.

1. Пользователь аутентифицируется, отправляя *AS\_Req*, и получает *AS\_Rep* ответ с сессионным ключом.
2. Далее он запрашивает *TGT* билет для обращения к сервису.
3. Он обращается к сервису для получения *TGS* билета. Потом *DC* видит, что у Сервиса А установлен флаг *TRUSTED\_FOR\_DELEGATION* и в ответ *DC* отправляет *TGS* билет с флагом *ok-as-delegate*.
4. Пользователь видит ответ и понимает, что ему необходимо передать свои данные Сервису А. Для этого пользователь снова обращается к *DC* для выдачи ему перенаправляемого *TGT*. *DC* выполняет необходимые проверки и отправляет пользователю *TGT* с правом передачи.
5. Пользователь отправляет немного расширенный запрос *AP\_Req*. Теперь помимо *TGS* билета там содержится перенаправляемый *TGT*.
6. Сервис А, расшифровав все данные запрашивает *TGS* билет для доступа к Сервису Б.

Для атаки необходима учетная запись пользователя с неограниченным делегированием.

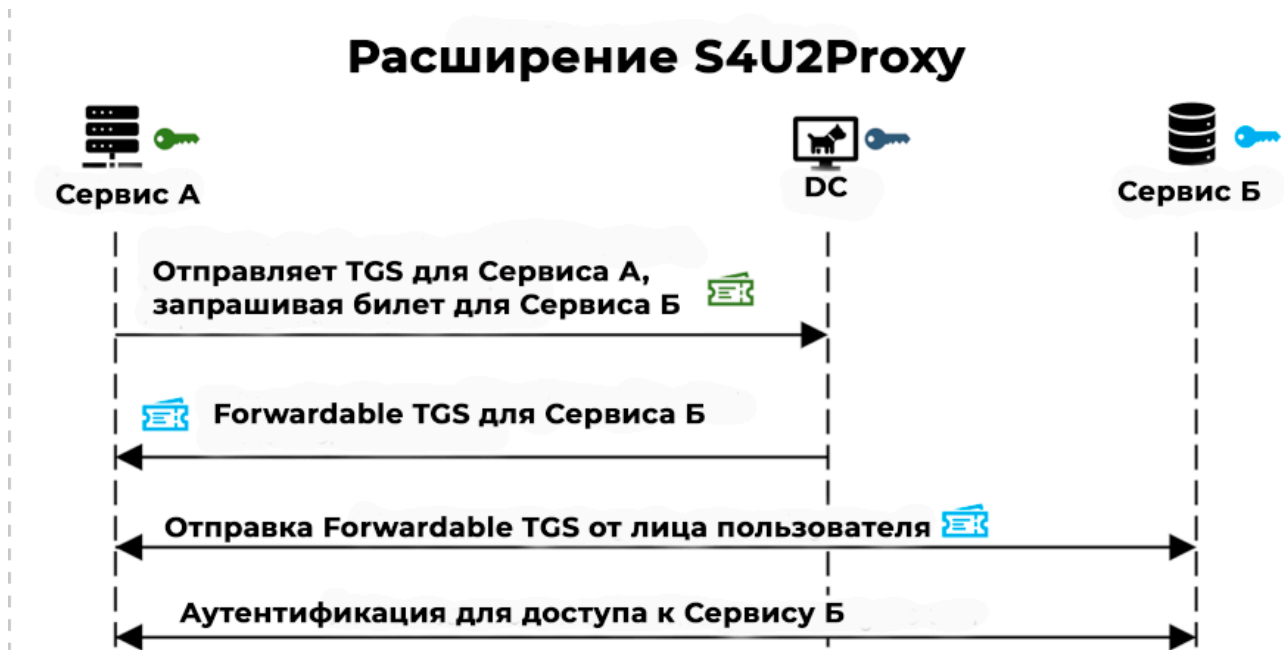
### **Ограниченном делегирование.**

Ограниченное делегирование появилось с целью снизить область делегирования и тем самым повысить защищенность. Для этого Microsoft выпустила 2 расширения: *S4U2Self* и *S4U2Proxy*.

### **Ограниченном делегирование с использованием только протокола Kerberos (*S4U2Proxy*).**

1. *User* стандартно получает *TGS*-билет с флагом *Forwardable* для доступа к Сервису А и отправляет его на указанный сервис.

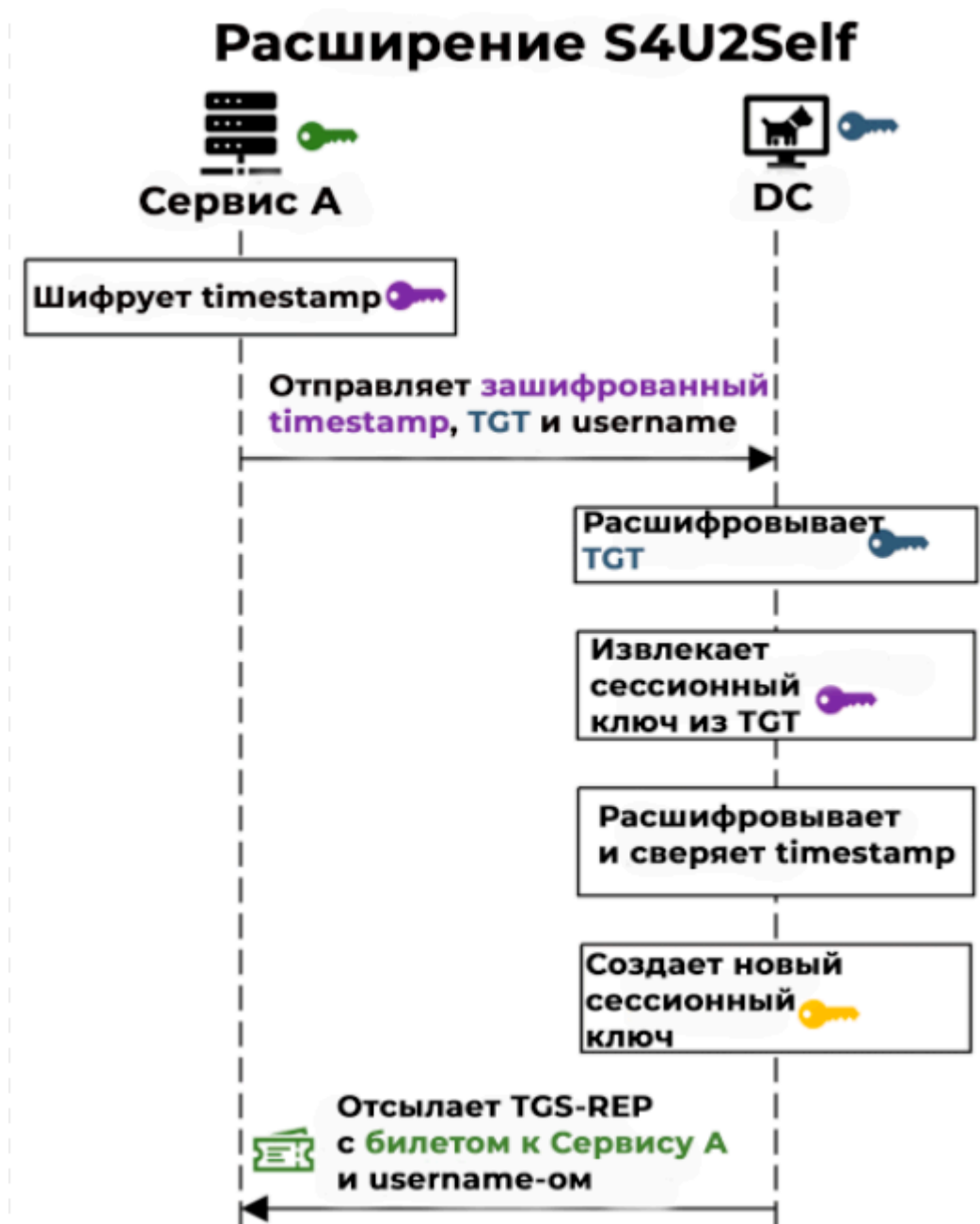
- Сервис А пересылает полученный *TGS*-билет на контроллер домена с целью получения доступа к Сервису Б от имени *User*. Как было замечено ранее, *TGS*-билет содержит имя пользователя для которого он предназначался. Таким образом Сервис А доказывает, что *User* действительно обращался к нему.
- Контроллер домена проверяет, что атрибут *msds-allowedtodelegateto* “Владельца А” содержит *SPN* Сервиса Б и в случае успешной проверки отправляет Сервису А *Forwardable TGS*-билет к Сервису Б для *User*.



#### Ограниченное делегирование со сменой протокола (*S4U2Self* и *S4U2Proxy*)

Расширение *S4U2Self* применяется если пользователю с ограниченным делегированием требуется пройти аутентификацию не через *Kerberos*. В данном случае может использоваться любой протокол, например *NTLM*.

- В начале пользователь проходит аутентификацию к Сервису А через любой протокол.
- Сервис А запрашивает у *DC* перенаправляемый *TGS* к самому себе.
- DC* проверяет, что у учетной записи в *UAC* установлен флаг *TRUSTED\_TO\_AUTH\_FOR\_AUTHENTICATION*. В результате успешной проверки *DC* отправляет Сервису А перенаправляемый *TGS* билет от имени пользователя к Сервису А.
- В дальнейшем для получения перенаправляемого *TGS* для Сервиса Б используется механизм *S4U2Proxy*.



### Ограниченное делегирование на основе ресурсов

*RBCD* (от англ. *Resource Based Constrained Delegation*)

Рассмотрим процесс *RBCD* по пунктам:

1. Каким образом *User* прошел аутентификацию к Сервису А неважно, допустим, что с использованием отличного от *Kerberos* протокола *NTLM*.
2. Сервис А обращается к контроллеру домену для получения *TGS*-билета “на себя” от имени пользователя *User*.

3. Контроллер домена проверяет, что у учетной записи «Владелец А» активен флаг *TRUSTED\_TO\_AUTH\_FOR\_DELEGATION*. Указанный флаг **не активен**, поэтому в ответ отправляется обычный *nonforwardable* TGS-билет без права передачи для *User* к Сервису А. Таким образом *S4U2Self* работает также, как и в случае классического ограниченного делегирования.
4. С использованием полученного TGS-домена для получения *Forwardable* TGS-билета к Сервису Б от имени *User*. Контроллер домена проверяет, что в атрибуте *msDS-AllowedToActOnBehalfOfOtherIdentity* учетной записи «Владелец Б» установлен *SID* учетной записи «Владелец А». Также контроллер домена проверяет, что «Владелец А» обладает хотя бы одним *SPN*. В результате контроллер домена отправляет в ответ *Forwardable* TGS-билет.

Указанное поведение является важной отличительной особенностью. При *RBCD S4U2Proxy* возвращает *Forwardable* TGS-билет при предоставлении *Nonforwardable* билета, в то время как при ограниченном делегировании *S4U2Proxy* возвращает *Forwardable* TGS-билет только при предоставлении *Forwardable* билета.

5. Сервис А обращается от имени *User* к Сервису Б с использованием полученного *Forwardable* TGS-билета.

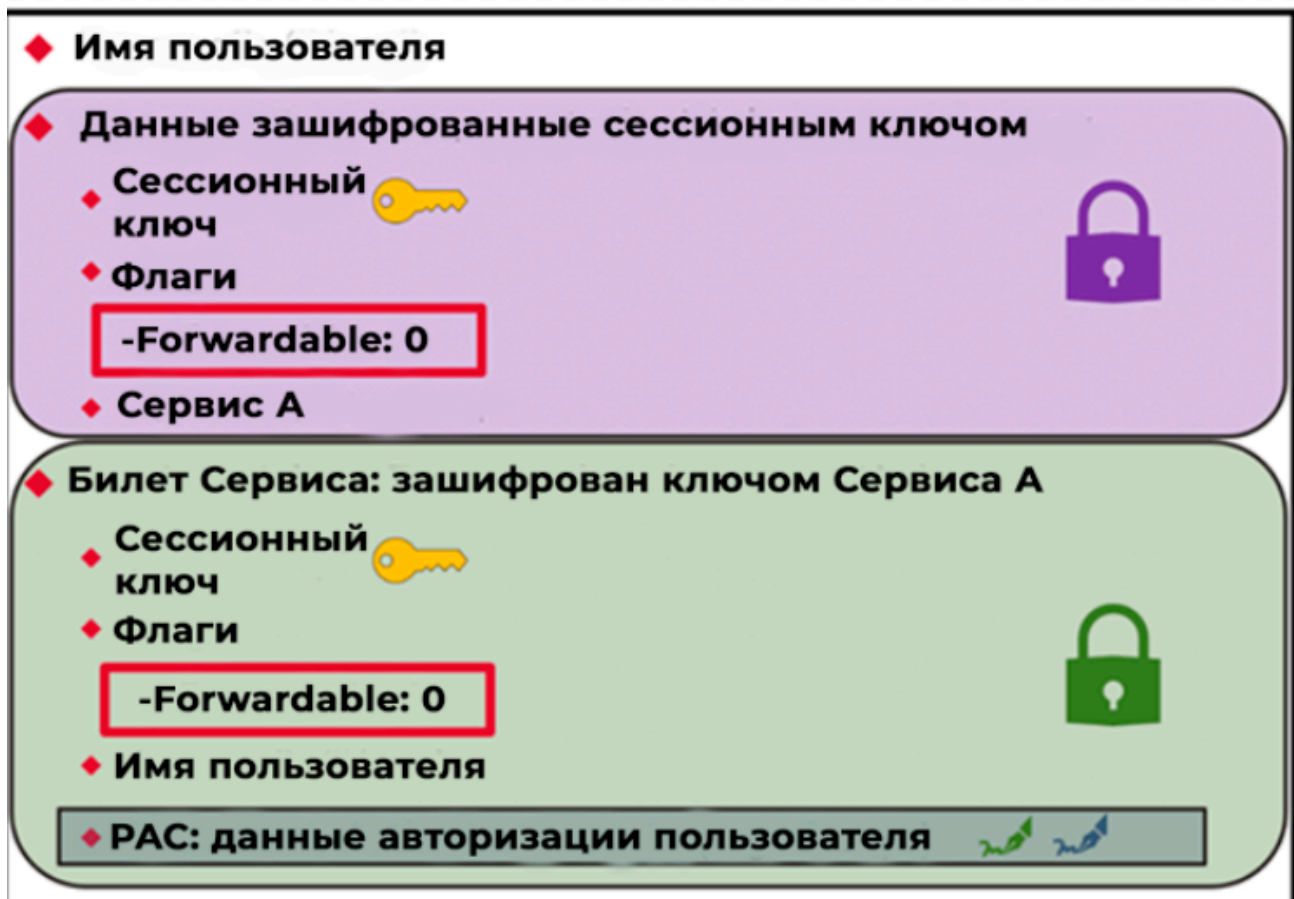
Еще раз отметим:

- *S4U2Self* при *RBCD* возвращает *Nonforwardable* TGS-билет.
- *S4U2Proxy* всегда в случае успеха возвращает *Forwardable* TGS-билет.

### **Амака Bronze Bit**

Давайте подробнее рассмотрим структуру данных *TGS\_REP*, возвращаемую *DC* после обмена *S4U2self*. Рассмотрим сценарий, в котором Сервис А не является “*TrustedToAuthForDelegation*” или указанный пользователь защищен от делегирования (поскольку он является членом группы “Защищенные пользователи” или настроен с параметром “Учетная запись конфиденциальна и не может быть делегирована”). Вот как мог бы выглядеть этот *TGS\_REP*:





Из-за установленных мер защиты флаг пересылки не установлен (т.е. его значение равно 0). Это означает, что заявка на обслуживание была бы отклонена, если бы использовалась в качестве доказательства при обмене S4U2proxу.

Посмотрите внимательно на то, где в ответе находится флаг пересылки. Флаг переадресации служебного билета зашифрован с помощью *Service A long-term*. Флаг пересылки отсутствует в подписанном PAC. Сервис А может свободно расшифровать, установить значение флага пересылки равным 1 и повторно зашифровать заявку на обслуживание. Поскольку его нет в подписанном PAC, DC не может обнаружить, что значение было изменено.

Мы преобразовали билет, не подлежащий пересылке, в билет, подлежащий пересылке. Этот переадресуемый сервисный билет может быть предоставлен в качестве подтверждения в *S4U2proxу exchange*, что позволяет нам делегировать аутентификацию Сервиса Б от имени любого пользователя по нашему выбору.